

The second advantage of COBOL is its legacy. During the sixties, when trained programmers were very rare, a simple language like COBOL was in high demand because of its shallow learning curve. At a time when computers were a scarce resource, it was economical to train people in a relatively easy-to-learn language like COBOL. The relatively large number of COBOL programmers naturally led to a lot of COBOL code and applications being developed. Thus, even today, a large number of well-written COBOL code is in existence. Financial institutions all over the world still have huge amounts of COBOL code being executed from powerful mainframes on a regular basis. The existence of such a large amount of legacy code makes COBOL still relevant.

Notice that nobody repairs a smoothly running car. Similarly, nobody is going to port the still smoothly running COBOL code into some other programming language, even if it is the latest in the programming world. I feel some of the claims by members of the COBOL programmer community are a bit exaggerated. My research for this article made me come across statements like 'more than 200 billion lines of COBOL code are still active', '80 per cent of all the code written is in COBOL', etc. I do not want to refute these claims as I don't know where to look for evidence to prove or disprove these statements. But I have heard similar remarks about C, C++, Java, Python, etc, and the math doesn't add up. Whatever be the case, there definitely is a really large COBOL code base still doing service to mankind. Data intensive operations are often carried out by mainframe computers. Thus, it is highly likely that you will see some COBOL code if you are working with mainframes.

I could also argue that there is a third point in favour of the relevance of COBOL. With even 5-year old children learning Python, what do you think is the average age of a Python programmer? I believe it to be in the early twenties. (No

sources to substantiate this claim, though.) What about the average age of a COBOL programmer? It could be well over forty, I believe. Again, it is just a guess. So here comes the third advantage of having familiarity with COBOL. One day you might be a member of a very small elite group that can maintain the large amount of COBOL legacy code. Like a traditional art form like Kathakali, which still attracts admirers and performers, COBOL will also attract young professionals to learn and code, time and again.

The history

Now let us learn some history. COBOL was designed in 1959 and its development was guided by a consortium called the Conference/Committee on Data Systems Languages (CODASYL) created for the development of a standard programming language for business processing. No single individual can be given the entire credit for the development of COBOL. But the name of one person must be mentioned, Grace Hopper. She was a pioneer in programming and one of the greatest programming language designers. As a side note, the illustrious career of Grace Hopper is inspiring and worth a few Internet searches to obtain more information. A programming language called FLOWMATIC developed by her has influenced the development of COBOL a lot.

Like all other programming languages, COBOL has also gone through a lot of revisions and upgrades over the years. Some of the standards of COBOL that made an impact include COBOL-60, COBOL-61, COBOL-65, COBOL-68, COBOL-74, COBOL-85, etc, with each introducing much awaited features. Finally, the long awaited object-oriented dialect of COBOL came in 2002, thus making COBOL a truly modern language. The latest standard of COBOL is COBOL 2014. Though COBOL is modernising almost as fast as any young programming language, it is still lagging behind them. A lot of

features are missing but none of them are critical to the domain in which COBOL applications are deployed. Though of the least concern, it was disheartening to know that COBOL does not have a mascot or a logo. May be that is something the COBOL programming community should come up with soon, and the earlier the better.

Running COBOL programs in Linux

The next important question is: how to run COBOL programs in Linux? An open source COBOL compiler called GnuCOBOL helps us in this aspect. GnuCOBOL was earlier known as OpenCOBOL. GnuCOBOL can be installed in Ubuntu with the command `sudo apt install open-cobol`. The installation is simple in other Linux distributions also.

Now let us see a simple COBOL program so that we can test our compiler. Consider the program `cob1.cob` shown below. It is the classical program of printing a message on the screen. COBOL programs can also have the extensions `.cbl` and `.cpy`:

```
id division.
program-id. cob1.
*> THIS IS A COMMENT IN COBOL
procedure division.
display "Doesn't COBOL look like
English?"
stop run.
```

Let us go through the code for better understanding. First, let me make a comment. Don't you feel that COBOL programs look like English text? I thought about showing one more COBOL program that almost looks like English prose but then decided not to do that. This lack of mathematical notations might be a hindrance to the mathematically oriented programmers of today but this was not so in the sixties, seventies and eighties where the bulk of the COBOL code was produced.